

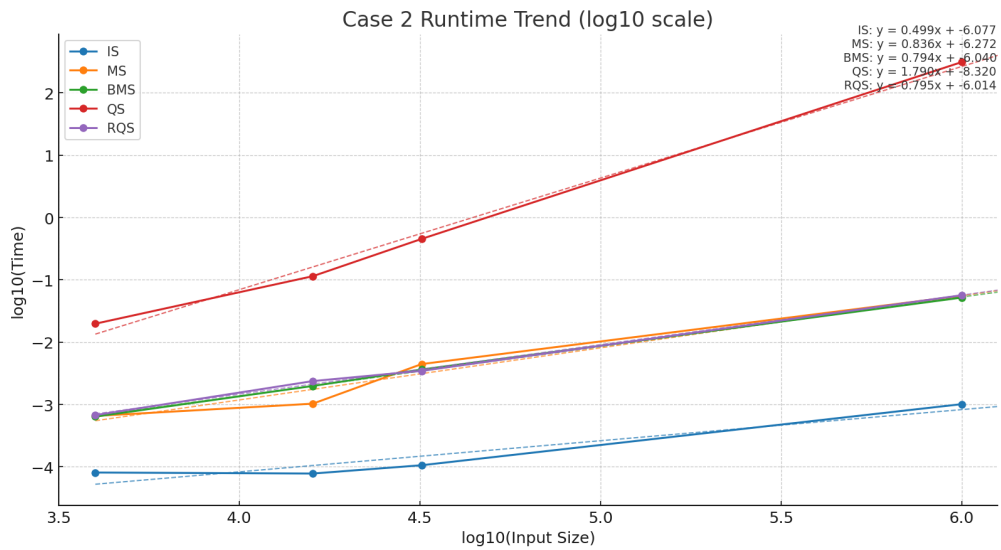
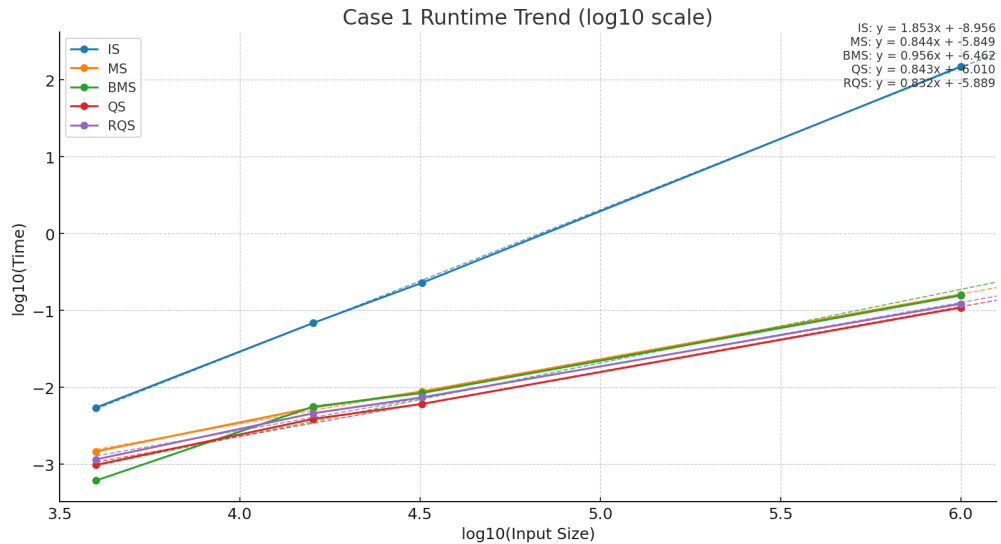
Algorithm PA1 Report

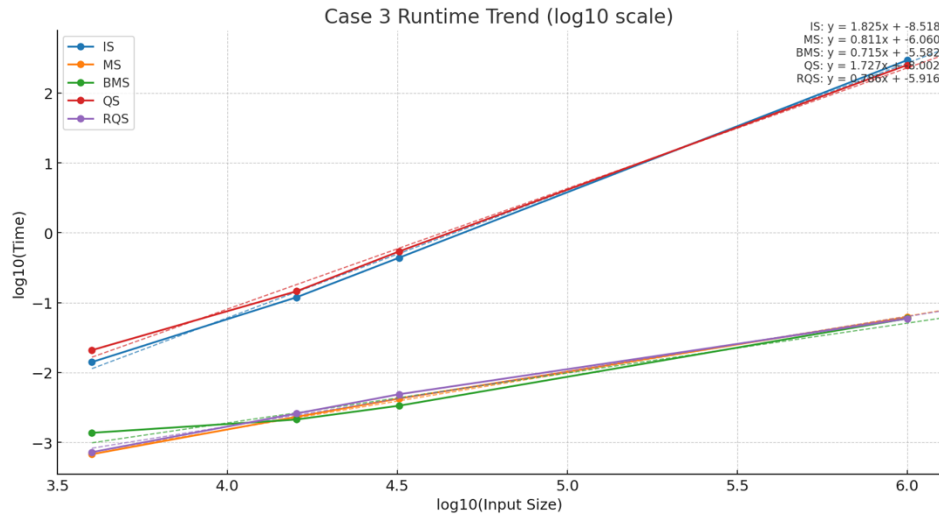
B13901104 劉祐丞

1. Table of run time and memory usage : run on EDA union

Input Size	IS		MS		BMS		QS		RQS	
	CPU time(s)	Memory(KB)	CPU time(ms)	Memory(KB)	CPU time(s)	Memory(KB)	CPU time(s)	Memory(KB)	CPU time(s)	Memory(KB)
4000.case2	0.000081	6076	0.000655	6076	0.000639	6076	0.019757	6200	0.000675	6076
4000.case3	0.014079	6076	0.000675	6076	0.001366	6076	0.02086	6076	0.000722	6076
4000.case1	0.005461	6076	0.001469	6076	0.000614	6076	0.000978	6076	0.001154	6076
16000.case2	0.000078	6228	0.00103	6228	0.001983	6228	0.114353	7108	0.002373	6228
16000.case3	0.119247	6228	0.002316	6228	0.00212	6228	0.145309	6604	0.002586	6228
16000.case1	0.06879	6228	0.005454	6228	0.00561	6228	0.003877	6228	0.004601	6228
32000.case2	0.000106	6360	0.004446	6360	0.003634	6360	0.453082	8176	0.003462	6360
32000.case3	0.438457	6360	0.004226	6360	0.003339	6360	0.539343	7156	0.004865	6360
32000.case1	0.228187	6360	0.008817	6360	0.008429	6360	0.006067	6360	0.007394	6360
1000000.case2	0.001012	12316	0.055529	14176	0.051847	14176	310.861	72640	0.05655	12316
1000000.case3	295.771	12316	0.061621	14176	0.05928	14176	251.853	33184	0.058992	12316
1000000.case1	149.234	12316	0.161336	14176	0.156972	14176	0.109013	12316	0.122056	12316

2. Trending Plot





3. Analysis

a. 斜率分析

因為 test case 介在 4000~100000 之間，因此若 time complexity 為 $n \log n$ ，斜率大約會是 1.2。而 n^2 和 n 的斜率則會是 2 和 1。

下表是五種演算法在 case 1, 2, 3 的理論複雜度（average, sorted, reverse sorted）。

	IS	MS	BMS	QS	RQS
Case 1	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Case 2	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n \log n)$
Case 3	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n \log n)$

從 trend line 可以看出測量出的斜率略低於理論斜率 0.3~0.5，例如 IS 的 case 2 斜率只有 0.5（理論上是 1），QS 的 case 2 斜率只有 1.8（理論上是 2），而 MS 的 case 2 斜率只有 0.8（理論上是 1.2）。

b. 偏差原因分析

我認為主要原因是測資的 n 不夠大且樣本點不足。在我們的測試中，最大的測資大小只到 $n = 1e6$ ，因此其他常數項和線性項對結果影響很顯著。例如 $T(n) = n^2 + 1000n$ ，則對於 $n=4000$ 而言，線性項的貢獻是平方項的 0.25 倍，對結果的影響非常大。因此要得到更正確的理論斜率，測量 $n=10000, 33000, 100000, 330000, 1000000$ 可能可以得到更精準的結果。此外因為樣本點少，小測資的偏移對斜率的影響大，例如從 trend line 中很明顯可以看出 $n=4000$ 的測資明顯上凹，影響正確的斜率表現。

c. MS 和 BMS 的差異分析

這兩個演算法其實速度差異不大，無論是看執行時間的數字還是幾乎疊在一起的 trend line。但其實仔細觀察數據，幾乎所有的測資上 MS 都略慢於 BMS。我認為原因在切割方式，MS 的切割使用遞迴，因此會有深度 $\log n$ 的 function call stack，而 function call 和 stack maintenance 相較 iteration 更花時間，且記憶體讀取效率更差（資料非連續），因此整體而言 MS 更慢一些。

d. QS 和 RQS 的差異分析

QS 和 RQS 差異在 pivot 的選擇，QS 是固定選最後一個，而 RQS 則是隨機在陣列中選擇一個。QS 的理想狀況是每次可以切割成數量相近的兩邊（比 pivot 大和比 pivot 小的元素數量接近），因此 recursion 深度是 $\log n$ ，但對於 sorted 或是 reverse sorted 的陣列而言，每此切割僅能切出一個元素，因而導致 n 的遞迴深度和 n^2 的時間複雜度。但 RQS 選擇方式是隨機的，因此選到 pivot 很大或很小的情況機率很低，因此平均而言執行時間是 $\Theta(n \log n)$ ，不會出現非常糟的執行狀況。

但對於非 worst case 而言，RQS 需要花時間做隨機，且隨機選出的 pivot 不見得好，因此從 case 1 的執行時間而言，可以看出 average case 下 RQS 不見得會比 QS 快。

e. 使用的資料結構和其他發現

這次 PA 主要用到的資料結構只有 vector，一種處理好所有記憶體分配、增刪元素等的陣列。使用 STL 容器可以很方便的利用 API 做到複製，也不需要自己分配管理記憶體和指標操作，真的非常方便。

這次比較特別的發現是針對 merge 的改進。一開始我使用的 merge 是 push_back 一個 INT_MAX 作為 sentinel，和課本一樣。但後來看到 hidden case 的測資範圍是 int，就改成先 merge 直到一個 subvector 已經沒有元素了，再把另一個 array 剩下的元素塞進去的做法。此外，在複製出 subvector 時，一開始我是開兩個空 vector 再一個一個 push_back，後來改成用傳入 iterator 的方法開 vector，就發現執行時間減少到原來的 80%，可見在頻繁擴大 vector 的情況下，一再的 reallocate 非常耗時，應該一開始就要預留好空間。